



TRAITEUR ARTISANAL BORDELAIS

# Documentation de déploiement

Guide complet du déploiement de la plateforme Vite & Gourmand en production : architecture d'hébergement, procédures d'installation, configuration serveur, sécurisation et maintenance opérationnelle.

**Version du document :** 2.0 · **Date :** 23 juillet 2026

Auteur : Laurent MARC

---

# Sommaire

---

|                                       |           |
|---------------------------------------|-----------|
| <b>1. Introduction et périmètre</b>   | <b>03</b> |
| 1.1 Objectifs du document             | 03        |
| 1.2 Environnements ciblés             | 03        |
| 1.3 Public visé                       | 04        |
| <b>2. Architecture d'hébergement</b>  | <b>05</b> |
| 2.1 Choix de l'hébergeur (Alwaysdata) | 05        |
| 2.2 Schéma d'infrastructure           | 06        |
| 2.3 Adressage et noms de domaine      | 07        |
| <b>3. Prérequis techniques</b>        | <b>08</b> |
| 3.1 Versions logicielles              | 08        |
| 3.2 Comptes et accès nécessaires      | 09        |
| 3.3 Variables d'environnement         | 09        |
| <b>4. Procédure de déploiement</b>    | <b>11</b> |
| 4.1 Configuration Alwaysdata          | 11        |
| 4.2 Déploiement du back-end Symfony   | 12        |
| 4.3 Configuration base PostgreSQL     | 14        |
| 4.4 Déploiement du front-end          | 15        |
| 4.5 Configuration Apache et .htaccess | 16        |

## **5. Sécurisation** **18**

---

5.1 Certificat HTTPS Let's Encrypt 18

5.2 Protection des secrets 18

5.3 Sauvegarde de la base de données 19

## **6. Workflow Git et mise en production** **20**

---

6.1 Branches main et develop 20

6.2 Procédure de mise à jour 21

6.3 Rollback en cas d'incident 22

## **7. Retour d'expérience** **23**

---

7.1 Incidents rencontrés et solutions 23

7.2 Bonnes pratiques identifiées 24



# 1. Introduction et périmètre

*Objectifs du document, environnements couverts et public visé.*

## 1.1 Objectifs du document

Cette documentation décrit l'ensemble des procédures nécessaires au déploiement et à la maintenance de la plateforme Vite & Gourmand en environnement de production. Elle permet à un développeur ou administrateur système de :

- Comprendre l'architecture d'hébergement retenue et ses justifications
- Reproduire le déploiement initial de A à Z sur un environnement neuf
- Mettre à jour l'application en production selon un workflow Git structuré
- Sécuriser les accès et les données (HTTPS, secrets, sauvegardes)
- Diagnostiquer et résoudre les incidents courants

Elle est le résultat de l'expérience concrète de déploiement du projet et intègre les enseignements tirés des incidents rencontrés lors de la mise en production.

## 1.2 Environnements ciblés

Le projet Vite & Gourmand utilise deux environnements distincts :

| Environnement        | Rôle                                                          | Emplacement                                             |
|----------------------|---------------------------------------------------------------|---------------------------------------------------------|
| <b>Développement</b> | Développement local, tests unitaires, itérations rapides      | Poste de travail du développeur (Windows + WSL2 Ubuntu) |
| <b>Production</b>    | Application accessible aux utilisateurs finaux et au jury ECF | Alwaysdata (datacenter Paris)                           |

Le projet ne dispose pas d'environnement de staging intermédiaire — un choix assumé compte tenu du périmètre pédagogique. Les tests fonctionnels sont réalisés en développement puis directement en production après validation.

**Note sur les évolutions futures :** pour un usage commercial réel, l'ajout d'un environnement de staging identique à la production serait indispensable, permettant de valider les mises à jour sur une réplique de la base de production avant déploiement final.

## 1.3 Public visé

Ce document s'adresse à trois profils :

- **L'auteur du projet** — comme aide-mémoire pour les mises à jour futures
- **Le jury ECF DWWM Studi** — pour évaluer la maîtrise des compétences de déploiement
- **Tout futur mainteneur** — pour reprendre le projet et l'exploiter en toute autonomie

La compréhension de ce document nécessite des connaissances de base en administration système Linux, en SSH, en Git et en PHP/Symfony.

## 2. Architecture d'hébergement

---

*Choix de l'hébergeur, schéma d'infrastructure et adressage.*

### 2.1 Choix de l'hébergeur (Alwaysdata)

L'hébergeur retenu est **Alwaysdata**, une société française basée à Paris opérant depuis 2006. Ce choix a été motivé par plusieurs critères :

#### Souveraineté et conformité

- Datacenters situés en France (Paris)
- Société de droit français, soumise au RGPD nativement
- Pas de transfert de données hors Union européenne

#### Compatibilité technique

- Support natif de PHP 8.2+ avec choix de version par site
- PostgreSQL 15+ disponible (PostgreSQL 17 en production)
- Extensions PHP courantes préinstallées (pdo\_pgsql, mbstring, curl, intl, xml, zip)
- Accès SSH complet avec Composer, Git et outils standards
- Certificats HTTPS Let's Encrypt automatiques

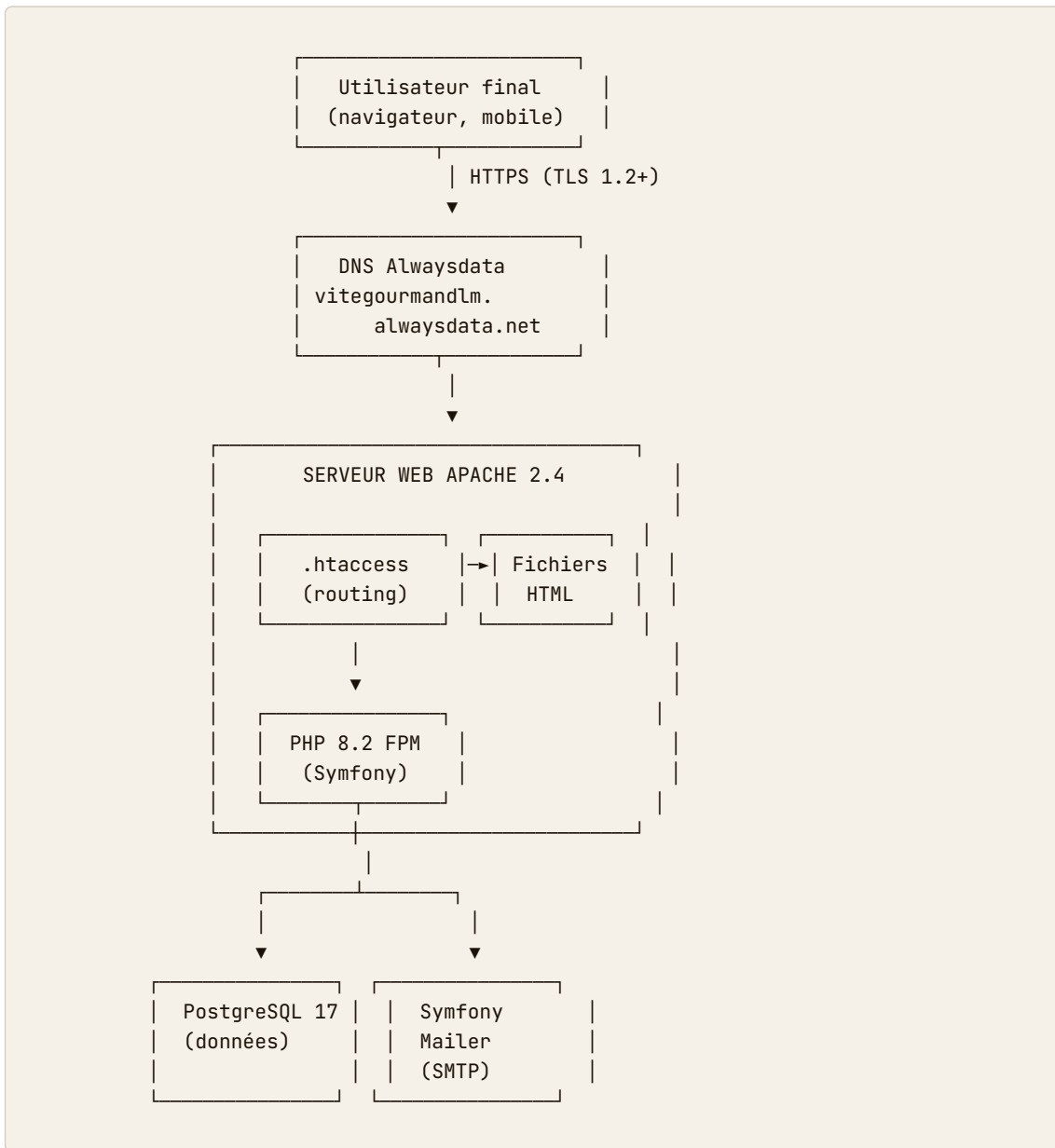
#### Offre adaptée au périmètre ECF

- Offre gratuite « Public Cloud » suffisante pour la démonstration ECF
- 256 Mo de RAM et 100 Mo de disque partagés (offre gratuite)
- Bande passante illimitée
- Migration vers des offres payantes (2 €/mois à 32 €/mois) sans changement d'URL

#### Interface d'administration

L'espace client Alwaysdata ( [admin.alwaysdata.com](https://admin.alwaysdata.com) ) propose une interface complète pour gérer les sites, bases de données, tâches planifiées, adresses e-mail et journaux d'accès. Ce niveau de contrôle facilite l'exploitation autonome sans nécessiter de connaissance approfondie en administration système.

### 2.2 Schéma d'infrastructure



L'architecture reste volontairement simple, en cohérence avec le périmètre pédagogique du projet. Elle est cependant conçue pour évoluer vers des configurations plus complexes (load balancer, cache Redis, CDN) si le trafic le justifiait.

## 2.3 Adressage et noms de domaine



| Composant        | Adresse                                               | Description                              |
|------------------|-------------------------------------------------------|------------------------------------------|
| Site public      | <code>https://vitegourmandlm.alwaysdata.net</code>    | URL principale accessible au public      |
| Accès SSH        | <code>ssh-vitegourmandlm.alwaysdata.net</code>        | Point d'entrée SSH pour l'administration |
| Base PostgreSQL  | <code>postgresql-vitegourmandlm.alwaysdata.net</code> | Serveur de base de données               |
| Panel Alwaysdata | <code>https://admin.alwaysdata.com</code>             | Interface d'administration               |

**Évolution vers un nom de domaine personnalisé :** pour un usage commercial, un domaine `vitegourmand.fr` pourrait être acquis auprès d'un registrar (OVH, Gandi) puis pointé vers Alwaysdata via une redirection DNS de type CNAME. Le certificat Let's Encrypt serait automatiquement délivré pour le nouveau nom de domaine.

## 3. Prérequis techniques

*Versions logicielles, comptes nécessaires et variables d'environnement.*

### 3.1 Versions logicielles

| Composant             | Version minimale | Version en production |
|-----------------------|------------------|-----------------------|
| PHP (web)             | 8.2              | 8.2                   |
| PHP (CLI, migrations) | 8.2              | 8.4                   |
| Composer              | 2.6              | 2.7                   |
| Symfony               | 7.0              | 7.1                   |
| PostgreSQL            | 15               | 17                    |
| Apache                | 2.4              | 2.4                   |
| Git                   | 2.30             | 2.40                  |

#### Extensions PHP requises

Les extensions suivantes doivent être activées côté PHP :

- `pdo_pgsql` — connexion PostgreSQL via PDO
- `mbstring` — manipulation de chaînes UTF-8
- `intl` — internationalisation (dates, nombres)
- `curl` — requêtes HTTP sortantes
- `xml` et `dom` — parsing XML et manipulation DOM
- `zip` — décompression d'archives
- `openssl` — chiffrement et TLS
- `ctype` — validation de types de caractères
- `iconv` — conversion d'encodages

Sur Alwaysdata, ces extensions sont activées par défaut sur les versions PHP 8.2+.

### 3.2 Comptes et accès nécessaires

## Compte Alwaysdata

Le compte principal est nommé `vitegourmandlm`. Il donne accès à :

- Espace disque du compte : `/home/vitegourmandlm/`
- Racine web du site : `~/www/vitegourmand/public/`
- Accès SSH par clé publique (recommandé) ou mot de passe

## Compte GitHub

Le dépôt Git est hébergé sur GitHub à l'adresse :

`https://github.com/LaurentMARCdev/ECR\_Studi\_Site\_vite-et-gourmand`

Le dépôt est public pour permettre au jury et au tuteur de consulter le code.

## Utilisateur PostgreSQL

Un utilisateur applicatif dédié `vitegourmandlm_app` est créé pour la connexion depuis Symfony. Cet utilisateur dispose uniquement des droits DML (SELECT, INSERT, UPDATE, DELETE) sur les tables du schéma, et pas des droits DDL (CREATE, ALTER, DROP) qui sont réservés à l'utilisateur d'administration utilisé pour les migrations.

## 3.3 Variables d'environnement

Les variables d'environnement de production sont stockées dans un fichier `.env.local` à la racine du projet Symfony, en dehors du dossier public accessible par HTTP.

### Fichier `.env.local` (production)

```
# Environnement
APP_ENV=prod
APP_DEBUG=0
APP_SECRET=

# Base de données PostgreSQL
DATABASE_URL="postgresql://vitegourmandlm_app:@postgresql-vitegourmandlm.alwaysdata.net:5432/vitegourmandlm"

# Envoi d'e-mails
MAILER_DSN=smtp://:@smtp-vitegourmandlm.alwaysdata.net:465?encryption=ssl
MAILER_FROM=contact@vitegourmand.fr

# CORS (nelmio/cors-bundle)
CORS_ALLOW_ORIGIN='^https?://(vitegourmandlm\.alwaysdata\.net)$'

# Rate limiter
LOCK_DSN=flock
```

**Sécurité** : le fichier `.env.local` ne doit **jamais** être committé sur Git. Il est exclu par `.gitignore` . Sur le serveur, ses permissions doivent être restreintes ( `chmod 600` ) pour empêcher toute lecture par d'autres utilisateurs du système.

## Génération de APP\_SECRET

La variable `APP_SECRET` est une chaîne aléatoire utilisée par Symfony pour signer les cookies de session, générer les tokens CSRF et chiffrer certaines données sensibles. Elle doit être unique pour chaque déploiement et jamais partagée :

```
php -r "echo bin2hex(random_bytes(16));"
```

Cette commande génère une chaîne hexadécimale de 32 caractères à copier dans `APP_SECRET` .

# 4. Procédure de déploiement

---

*Étapes détaillées pour un déploiement complet, de la création du compte à la mise en ligne.*

## 4.1 Configuration Alwaysdata

### 1 Créer un compte Alwaysdata

Se rendre sur `https://www.alwaysdata.com` et souscrire à l'offre gratuite « Public Cloud ». Le nom du compte devient un sous-domaine (dans notre cas : `vitegourmandlm.alwaysdata.net`).

### 2 Créer le site web

Dans le panel Alwaysdata, section *Web* → *Sites*, cliquer sur « Ajouter un site ».  
Choisir :

- Type : PHP
- Version PHP : 8.2
- Chemin racine : `/www/vitegourmand/public/`
- Adresses : `vitegourmandlm.alwaysdata.net`

### 3 Créer la base PostgreSQL

Dans *Bases de données* → *PostgreSQL*, cliquer sur « Ajouter une base ». Choisir le nom `vitegourmandlm_vg`, créer un utilisateur `vitegourmandlm_app` avec un mot de passe fort et lui accorder tous les droits sur cette base.

### 4 Activer l'accès SSH

Dans *Environnement* → *SSH*, activer le service. Ajouter votre clé publique SSH pour l'authentification par clé (recommandé pour la sécurité).

### 5 Configurer les e-mails sortants

Dans *E-mails* → *Adresses*, créer `contact@vitegourmand.fr` (ou une adresse équivalente sur votre domaine). Récupérer les informations SMTP pour la variable `MAILER_DSN`.

## 4.2 Déploiement du back-end Symfony

### Étape 1 — Cloner le dépôt Git

Se connecter en SSH sur le serveur puis récupérer le code source :

```
ssh vitegourmandlm@ssh-vitegourmandlm.alwaysdata.net
cd ~
git clone https://github.com/LaurentMARCdev/ECR_Studio_Site_vite-et-gourmand.git v
cd www/vitegourmand-source
git checkout main
```

## Étape 2 — Installer les dépendances Composer

Composer est déjà installé sur Alwaysdata. En production, on installe uniquement les dépendances de production :

```
composer install --no-dev --optimize-autoloader
```

L'option `--no-dev` évite d'installer les paquets de développement (PHPUnit, Symfony Maker, etc.), et `--optimize-autoloader` génère un autoloader optimisé pour de meilleures performances.

## Étape 3 — Créer le fichier .env.local

Copier le contenu du fichier .env.local depuis un environnement sécurisé (jamais depuis Git) et le placer à la racine du projet :

```
nano .env.local
# Coller le contenu avec les valeurs de production
chmod 600 .env.local
```

## Étape 4 — Vider le cache Symfony

```
php bin/console cache:clear --env=prod --no-debug
php bin/console cache:warmup --env=prod
```

## Étape 5 — Exécuter les migrations Doctrine

```
php bin/console doctrine:migrations:migrate --no-interaction --env=prod
```

## Étape 6 — Charger les référentiels initiaux

Un script SQL `seed-referentiels.sql` est fourni pour peupler les tables de référence (thèmes, régimes, allergènes, rôles) :

```
psql -h postgresql-vitegourmandlm.alwaysdata.net \  
-U vitegourmandlm_app \  
-d vitegourmandlm_vg \  
-f scripts/seed-referentiels.sql
```

## Étape 7 — Créer les comptes de démonstration

Un second script `seed-demo-users.sql` crée les trois comptes de démonstration (client, employé, admin) avec des mots de passe hashés Argon2id :

```
psql -h postgresql-vitegourmandlm.alwaysdata.net \  
-U vitegourmandlm_app \  
-d vitegourmandlm_vg \  
-f scripts/seed-demo-users.sql
```

## 4.3 Configuration base PostgreSQL

### Vérification de la connexion

Tester la connexion à la base depuis le serveur :

```
psql -h postgresql-vitegourmandlm.alwaysdata.net \  
-U vitegourmandlm_app \  
-d vitegourmandlm_vg -c "SELECT version();" 
```

Doit retourner la version PostgreSQL installée (17.x en production).

### Vérification des tables créées

```
psql -h postgresql-vitegourmandlm.alwaysdata.net \  
-U vitegourmandlm_app \  
-d vitegourmandlm_vg -c "\dt"
```

Doit lister les 17 tables du schéma (utilisateur, menu, plat, commande, avis, etc.).

### Configuration recommandée

Sur les offres payantes Alwaysdata, il est possible d'ajuster certains paramètres PostgreSQL :

- `max_connections` : nombre de connexions simultanées (défaut : 100)
- `shared_buffers` : cache de données en mémoire
- `work_mem` : mémoire par requête pour les tris et jointures

Sur l'offre gratuite, ces paramètres sont fixés et non modifiables. Ils restent suffisants pour la démonstration ECF.

## 4.4 Déploiement du front-end

Le front-end est constitué de fichiers HTML statiques enrichis avec Tailwind CSS et Alpine.js chargés via CDN. Aucune étape de build n'est nécessaire.

### Emplacement des fichiers

Les fichiers HTML doivent être placés directement dans le dossier racine du site Apache :

```
~/www/vitegourmand/public/  
├─ index.html  
├─ menus.html  
├─ menu-detail.html  
├─ commande.html  
├─ connexion.html  
├─ mon-espace.html  
├─ contact.html  
├─ employe.html  
├─ admin.html  
├─ mentions-legales.html  
├─ cgw.html  
├─ declaration-accessibilite.html  
├─ .htaccess  
└─ media/  
    └─ menus/  
        └─ (images des plats et menus)
```

### Transfert depuis le poste local

Utiliser SCP pour transférer les fichiers depuis le poste de développement :

```
scp front/*.html \  
vitegourmandlm@ssh-vitegourmandlm.alwaysdata.net:~/www/vitegourmand/public/
```

## 4.5 Configuration Apache et .htaccess

Le fichier `.htaccess` à la racine du site gère plusieurs points essentiels :

- Redirection des URLs propres vers les fichiers HTML correspondants
- Fallback vers l'API Symfony pour les routes `/api/*`
- Forçage du HTTPS via redirection 301
- En-têtes de sécurité HTTP



## Contenu du fichier .htaccess

```
RewriteEngine On

# Forçage HTTPS
RewriteCond %{HTTPS} !=on
RewriteRule ^(.*)$ https://%{HTTP_HOST}%{REQUEST_URI} [L,R=301]

# Routing URLs propres
RewriteRule ^menus/?$ /menus.html [L]
RewriteRule ^menus/[0-9]+/?$ /menu-detail.html [L]
RewriteRule ^commande/?$ /commande.html [QSA,L]
RewriteRule ^connexion/?$ /connexion.html [L]
RewriteRule ^mon-espace/?$ /mon-espace.html [L]
RewriteRule ^contact/?$ /contact.html [L]
RewriteRule ^employe/?$ /employe.html [L]
RewriteRule ^admin/?$ /admin.html [L]
RewriteRule ^mentions-legales/?$ /mentions-legales.html [L]
RewriteRule ^cgv/?$ /cgv.html [L]
RewriteRule ^declaration-accessibilite/?$ /declaration-accessibilite.html [L]

# Fallback API Symfony
RewriteCond %{REQUEST_FILENAME} !-f
RewriteCond %{REQUEST_FILENAME} !-d
RewriteRule ^api/(.*)$ /index.php [QSA,L]

# En-têtes de sécurité
Header set X-Content-Type-Options "nosniff"
Header set X-Frame-Options "DENY"
Header set Referrer-Policy "strict-origin-when-cross-origin"
```

**Note sur les URLs propres :** ce système permet aux visiteurs d'accéder à `/menus/1` au lieu de `/menu-detail.html?id=1`, ce qui améliore l'expérience utilisateur et le référencement naturel (SEO). Le parsing de l'ID est ensuite effectué côté JavaScript à partir de `window.location.pathname`.

# 5. Sécurisation

---

*Certificat HTTPS, protection des secrets et sauvegardes.*

## 5.1 Certificat HTTPS Let's Encrypt

Alwaysdata propose l'activation automatique d'un certificat Let's Encrypt en un clic.

### Procédure d'activation

1. Dans le panel Alwaysdata, aller dans *Web* → *Sites*
2. Éditer le site `vitegourmandlm.alwaysdata.net`
3. Section *SSL*, cocher « Activer le SSL »
4. Choisir « Let's Encrypt » comme fournisseur
5. Cocher « Forcer HTTPS » (redirection automatique HTTP → HTTPS)
6. Enregistrer

Le certificat est délivré en quelques secondes et renouvelé automatiquement tous les 90 jours par Alwaysdata, sans intervention manuelle.

### Vérification

Après activation, tester avec l'outil SSL Labs :

`https://www.ssllabs.com/ssltest/analyze.html?d=vitegourmandlm.alwaysdata.net`

Une note A ou A+ est attendue, avec support des protocoles TLS 1.2 et TLS 1.3 uniquement (les versions obsolètes SSL 3.0, TLS 1.0 et TLS 1.1 étant désactivées).

## 5.2 Protection des secrets

### Fichier `.env.local`

Ce fichier contient toutes les informations sensibles (mots de passe, URLs de base, secret d'application). Il doit être protégé par :

- Exclusion du dépôt Git via `.gitignore`
- Permissions restrictives : `chmod 600 .env.local`
- Emplacement hors du dossier public (à la racine du projet Symfony, pas dans `public/`)

## Clés SSH

L'authentification SSH par clé publique est privilégiée sur l'authentification par mot de passe :

```
# Générer une paire de clés localement
ssh-keygen -t ed25519 -C "vitegourmand-deploy"

# Copier la clé publique dans le panel Alwaysdata
# Environnement → SSH → Clés autorisées
```

## Gestion des mots de passe

Aucun mot de passe n'est stocké en clair dans le code. Les mots de passe utilisateurs sont hashés avec Argon2id avant stockage en base. Les mots de passe d'infrastructure (base PostgreSQL, SMTP) sont stockés uniquement dans `.env.local` côté serveur, et dans un gestionnaire de mots de passe côté développeur.

## 5.3 Sauvegarde de la base de données

### Sauvegarde automatique Alwaysdata

Alwaysdata effectue automatiquement des sauvegardes journalières de toutes les bases de données. Sur l'offre gratuite, l'historique est de 7 jours. Sur les offres payantes, il peut aller jusqu'à 30 jours.

### Sauvegarde manuelle

Pour effectuer une sauvegarde ponctuelle avant une mise à jour critique :

```
pg_dump -h postgresql-vitegourmandlm.alwaysdata.net \
-U vitegourmandlm_app \
-d vitegourmandlm_vg \
-f backup-$(date +%Y%m%d-%H%M).sql
```

### Restauration

En cas de besoin, restaurer une sauvegarde :

```
psql -h postgresql-vitegourmandlm.alwaysdata.net \
-U vitegourmandlm_app \
-d vitegourmandlm_vg \
-f backup-20260723-2200.sql
```

**Bonne pratique :** effectuer une sauvegarde manuelle systématiquement avant toute mise à jour incluant une migration Doctrine, afin de pouvoir rollback rapidement en cas d'incident.

# 6. Workflow Git et mise en production

*Organisation des branches, procédure de mise à jour et rollback.*

## 6.1 Branches main et develop

Le projet suit un workflow **GitFlow simplifié** avec deux branches principales et éventuellement des branches de fonctionnalité.

```
graph TD
    main["main (production stable)"]
    develop["develop (intégration continue)"]
    feature["feature/xxx feature/yyy hotfix/zzz  
(branches courtes, supprimées après merge)"]
    main -- "← merge après validation" --> develop
    develop -- "← merge des fonctionnalités" --> feature
```

### Rôle de chaque branche

- **main** — reflète l'état exact du site en production. Chaque commit sur main déclenche (théoriquement) un déploiement.
- **develop** — branche d'intégration où les fonctionnalités sont testées ensemble avant validation pour production.
- **feature/\*** — branches courtes pour développer une fonctionnalité isolée, mergées dans **develop** une fois validées.
- **hotfix/\*** — corrections urgentes de bugs en production, mergées directement dans **main** puis dans **develop**.

### Convention de nommage des commits

Les messages de commit suivent la convention **Conventional Commits** pour une lisibilité maximale :

```
feat: ajout du formulaire de commande
fix: correction du calcul de prix quand personnes > 500
docs: mise à jour du README
refactor: extraction de la logique de prix dans un service
chore: mise à jour des dépendances Composer
```

## 6.2 Procédure de mise à jour

### Étape 1 — Développement en local

Les développements se font sur la branche `develop` ou sur une branche `feature/*` :

```
git checkout develop
git checkout -b feature/nouveau-formulaire
# ... développement et tests locaux ...
git add .
git commit -m "feat: ajout du nouveau formulaire de commande"
git push origin feature/nouveau-formulaire
```

### Étape 2 — Merge dans develop

Une fois la fonctionnalité validée localement :

```
git checkout develop
git merge feature/nouveau-formulaire
git push origin develop
git branch -d feature/nouveau-formulaire
```

### Étape 3 — Validation et merge dans main

Après validation complète de `develop` (tests fonctionnels, revue de code) :

```
git checkout main
git merge develop
git push origin main
```

### Étape 4 — Déploiement en production

Se connecter en SSH sur le serveur et récupérer la dernière version :

```
ssh vitegourmandlm@ssh-vitegourmandlm.alwaysdata.net
cd ~/www/vitegourmand-source
git fetch --all
git checkout main
git pull origin main

# Installation des dépendances mises à jour
composer install --no-dev --optimize-autoloader

# Migrations éventuelles
php bin/console doctrine:migrations:migrate --no-interaction --env=prod

# Vidage du cache
php bin/console cache:clear --env=prod --no-debug

# Synchronisation des fichiers HTML modifiés
cp front/*.html ~/www/vitegourmand/public/
```

## 6.3 Rollback en cas d'incident

Si une mise à jour introduit un bug critique en production, la procédure de rollback est la suivante :

### Rollback du code

```
cd ~/www/vitegourmand-source
git log --oneline -10 # Identifier le commit précédent
git checkout

# Réinstallation dépendances et cache
composer install --no-dev --optimize-autoloader
php bin/console cache:clear --env=prod --no-debug
```

### Rollback de la base de données

Si une migration Doctrine est à l'origine du problème :

```
php bin/console doctrine:migrations:migrate prev --no-interaction --env=prod
```

Ou en cas de corruption plus grave, restaurer la sauvegarde manuelle effectuée avant la mise à jour (voir chapitre 5.3).

**Attention :** le rollback d'une migration peut entraîner une perte de données si la migration a créé de nouvelles colonnes ou tables contenant maintenant des données. Toujours prévoir cette possibilité avant de déployer une migration en production.

## **Communication d'incident**

En cas d'indisponibilité prolongée (plus de 15 minutes), une communication doit être faite auprès des utilisateurs actifs via :

- Une bannière d'information sur la page d'accueil
- Un e-mail transactionnel aux clients ayant une commande en cours
- Une mention sur les réseaux sociaux du traiteur



# 7. Retour d'expérience

*Incidents rencontrés lors du déploiement et bonnes pratiques identifiées.*

## 7.1 Incidents rencontrés et solutions

Le déploiement initial et les mises à jour successives ont révélé plusieurs difficultés dont la résolution enrichit désormais cette documentation.

### Incident 1 — Bundles Symfony manquants

**Symptôme** : après le premier `composer install --no-dev`, certaines pages retournaient une erreur 500 avec un message « Bundle X does not exist ».

**Cause** : certains bundles étaient déclarés uniquement en environnement `dev` dans `config/bundles.php`, alors qu'ils étaient utilisés en production.

**Solution** : vérification systématique du fichier `config/bundles.php` et retrait de la restriction `dev` pour les bundles nécessaires en production.

### Incident 2 — Configuration MAILER\_DSN manquante

**Symptôme** : les tentatives d'envoi d'e-mail (inscription, confirmation commande) échouaient silencieusement en production.

**Cause** : la variable `MAILER_DSN` n'avait pas été renseignée dans `.env.local`. Symfony utilisait alors le transport par défaut `null://` qui ignore les envois.

**Solution** : ajout de la variable `MAILER_DSN` avec les paramètres SMTP fournis par Alwaysdata, et vérification via un envoi de test après chaque déploiement.

### Incident 3 — Boucle de redirection sur /menus/{id}

**Symptôme** : l'accès à `/menus/1` provoquait une boucle infinie de redirections Apache.

**Cause** : la règle de rewriting dans `.htaccess` ne distinguait pas correctement les URLs propres des fichiers réels, provoquant une redirection récursive.

**Solution** : ajout de conditions `RewriteCond %{REQUEST_FILENAME} !-f` et `!-d` avant les règles, et utilisation de patterns plus précis ( `^menus/[0-9]+/?$` ).

#### Incident 4 — Permissions insuffisantes sur var/cache

**Symptôme** : Symfony ne parvenait pas à écrire dans le dossier cache, générant des erreurs 500 aléatoires.

**Cause** : après le `git clone`, le dossier `var/cache` n'existait pas et Symfony ne pouvait pas le créer avec les permissions par défaut.

**Solution** : création manuelle du dossier avec permissions étendues : `mkdir -p var/cache var/log && chmod -R 775 var/`

#### Incident 5 — Contenu des formulaires admin incohérent

**Symptôme** : lors de la création d'un menu depuis l'interface admin, les erreurs 422 remontaient sans détail exploitable.

**Cause** : le formulaire front envoyait des libellés en snake\_case alors que le DTO Symfony attendait des IDs en camelCase ( `themeIds`, `regimeIds`, `platIds` ).

**Solution** : refonte complète du formulaire pour aligner le format de sortie sur le contrat du DTO, avec conversion des libellés en IDs côté JavaScript avant envoi.

## 7.2 Bonnes pratiques identifiées

Ces incidents ont conduit à formaliser plusieurs bonnes pratiques applicables à tout projet Symfony déployé sur un hébergement mutualisé.

### Toujours tester en environnement proche de la production

Un environnement local avec PHP en mode `prod` et une base PostgreSQL identique à la production permet d'identifier tôt les problèmes de configuration liés à l'absence de dépendances `dev`.

### Automatiser autant que possible

Un script `deploy.sh` qui enchaîne les étapes ( `git pull`, `composer install`, migrations, `cache:clear` ) réduit les erreurs humaines et documente implicitement la procédure.

```
#!/bin/bash
# scripts/deploy.sh
set -e # Arrêt en cas d'erreur

echo "→ Récupération dernière version..."
git pull origin main

echo "→ Installation dépendances..."
composer install --no-dev --optimize-autoloader

echo "→ Migrations base de données..."
php bin/console doctrine:migrations:migrate --no-interaction --env=prod

echo "→ Vidage du cache..."
php bin/console cache:clear --env=prod --no-debug

echo "→ Synchronisation front..."
cp -f front/*.html ~/www/vitegourmand/public/

echo "✓ Déploiement terminé"
```

## Journaliser systématiquement

Symfony génère automatiquement des logs dans `var/log/prod.log`. Une consultation régulière (surtout après un déploiement) permet de détecter les warnings et erreurs qui n'atteignent pas l'utilisateur mais méritent correction.

## Documenter chaque incident

Chaque bug rencontré en production est une occasion d'améliorer la documentation. Le format « symptôme / cause / solution » utilisé ici est reproductible et facilite la lecture pour un futur mainteneur.

## Séparer strictement code et configuration

Le code (versionné avec Git) et la configuration (variables d'environnement dans `.env.local`) doivent rester rigoureusement séparés. Cela permet de déployer le même code sur plusieurs environnements sans modification, et de faire évoluer indépendamment code et configuration.

## Prévoir un plan de reprise d'activité

Documenter les procédures de rollback et de restauration *avant* d'en avoir besoin. Un plan de reprise testé une fois par trimestre garantit qu'il fonctionnera le jour où un incident majeur surviendra.